n = 83 K600 online frame: the rank stage's frame loop in the one-target online scan

2026-10-19

n = 83 K600 online frame: the rank stage's frame loop in the one-target online scan

Status (2026-10-19): protocol frozen, Tier 1 run pending. The protocol (frozen 2026-10-19T18:32:46Z) and the schedule are committed before any Tier 1 timing. The workflow is `.github/workflows/n83-onlineframe.yml` on branch `claude/n83-onlineframe`. Every number below that is not from an archived run comes from the cloud-agent VM. Those numbers are **local diagnostics**, not timing evidence (see `smoke/`).

Question

The n = 83 a = 1 K600 baseline is the rank-probe producer, `koblitz-n83-rankprobe-20261019` (PR #980, run 39214027289, producer digest `d4394df3...`). Its rank stage runs `KIC_RANK_SOLVE=frame`, the point-addition partner roots in the target's Frobenius frame, at 141.5 ns per probe. Its one-target **online** scan still runs the older S3-solve loop. On that run's 20 points it took 221.4 ns per probe, a mean of 1.37 M probes and 303.8 ms (online `S` 0.0117, online gap to tuned rho 0.129 [0.085, 0.183]).

The change is to run the online scan on the frame loop. Does that lower the **online** time on new public points, with every relation, probe count, root call, relation check and scalar unchanged? This is an iteration under AGENTS.md "Success criterion". The end-to-end cold cost is a pre-registered secondary. The expected change there is about 0.1 s of 166 s.

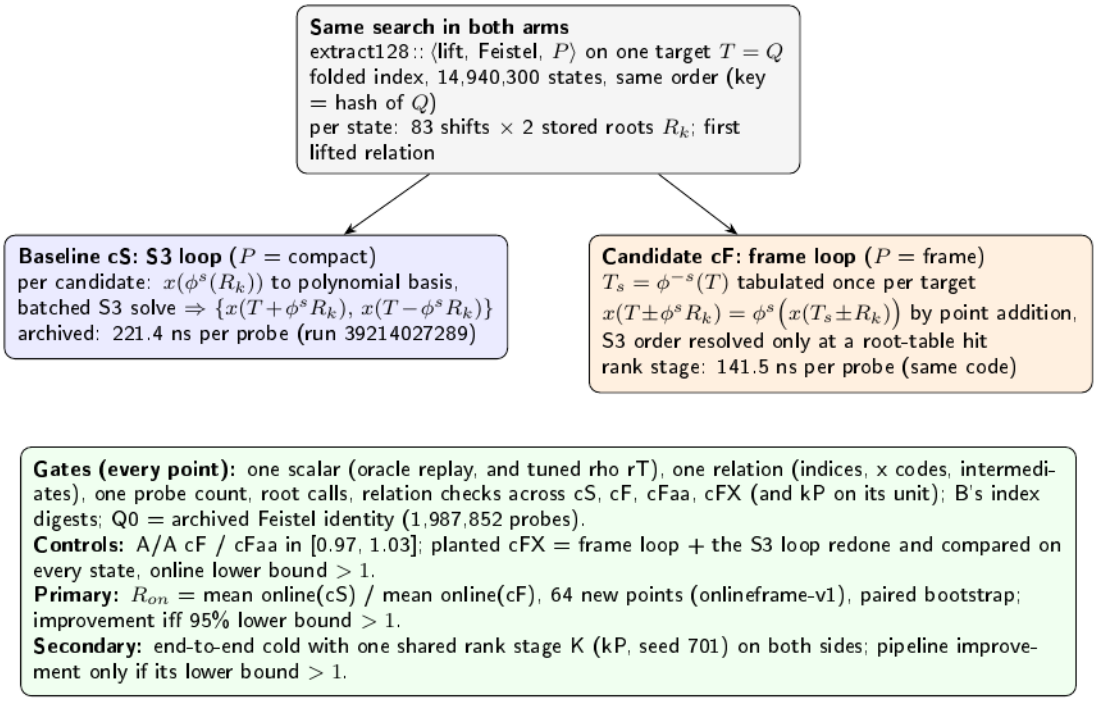


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

Is the online scan the same search as a rank attempt?

Yes. Both call `extract128::<CANON_LIFT,ORDER_FEISTEL,P>` on one target point T. The online scan uses T = Q. A rank attempt uses T = [a]G − R_j. Both scans share:

- the folded index (14,940,300 stored states) and the 16-byte compact table;
- the Feistel state order, keyed by the target's public hash;
- the candidates per state: 83 shifts × 2 stored roots, shift outer;
- the return at the first lifted relation.

Only the probe loop `P` differs:

- **Online, baseline (`PIPE_COMPACT`, `scan_state128_filtered`).** For every candidate it maps x(φ^s(R_k)) to the polynomial basis. It then solves the third summation polynomial S3(x(φ^s R_k), x(T), X) in a batch, which gives the two roots x(T ± φ^s R_k).
- **Rank stage (`PIPE_FRAME`, `scan_state128_frame`).** It tabulates T_s = φ^(−s)(T) once per target. It computes x(T_s ± R_k) by point addition from the stored root's own (x, y), with one batched inversion per state, and applies φ^s as a rotation of the normal-basis word. The S3 input is formed, and the S3 order resolved, only at a root-table hit.

So the frame trick applies directly. The online target is tabulated once, exactly as a rank attempt's target is.

The change: `KIC_ONLINE_SOLVE`

`make_online_source.py` derives the producer in two steps:

1. `make_rankprobe_source.py` is the rank-probe experiment's `make_online_source.py`, verbatim, with its five verbatim parents. Its output must hash to the rank-probe protocol's frozen digest `d4394df3...`, and the derivation checks this.
2. Seven counted, exact replacements in the wide module add `KIC_ONLINE_SOLVE`, which only the online target scan reads:
   - `s3` (the default, also when the variable is unset): the rank-probe producer's online scan, unchanged.

- `frame`: the online scan calls `extract128::<CANON_LIFT,ORDER_FEISTEL,PIPE_FRAME>`. This is the same function, with the same const parameters, that the rank stage runs under `KIC_RANK_SOLVE=frame`. No new arithmetic is involved.
- `frame_s3plant` (the planted positive control): on every state the scan reaches, it first redoes the S3 loop: the batched S3 solve of every candidate, its keys, and its whole probe loop, on a private probe counter. It then runs the frame loop and compares the two loops' outputs (a panic on any difference):
  - the relation, or none;
  - the probe counter, the root calls and the relation checks;
  - for every candidate, both keys (canonical word, normal-basis partner, shift, candidate index), up to pair order.

  The redone S3 work stays inside the online interval, and its result is discarded.

Each target record gains `online_solve` and `online_counters`: frames built, fallback states, exceptional inputs, resolved hits, swapped pairs, and the planted control's checked candidates, states and swapped pairs. The summary's rank counters are read before the online scan, so they stay the rank stage's own. Setup, the index, the rank stage and the `s3` online scan are the rank-probe producer's code, byte for byte. The frozen derived digest is `e6a944e6...`.

Why the output is exact

- E is defined over F_2, so $\varphi(x, y) = (x^2, y^2)$ is a group automorphism. Hence $T \pm \varphi\hat{}s(R) = \varphi^{s(\varphi}(-s)(T) \pm R)$. The two roots of S3(x($\varphi\hat{}$s R), x(T), X) are exactly x(T + $\varphi\hat{}$s R) and x(T − $\varphi\hat{}$s R).
- The frame loop computes these two field elements by point addition and then rotates them. Each candidate's partner words, canonical keys and shifts are therefore the S3 loop's, possibly in the other order. The probe positions are those of the same keys.
- The exceptional inputs are handled exactly as in the S3 loop. These are x(R_k) = x(T_s), x(R_k) = 0, x(T_s) = 0, and roots that fail the curve check. They go to the S3 solver.
- At a hit, the frame loop solves that one candidate's S3 input. It then evaluates the pair in the S3 order, at the S3 loop's probe positions. So the first lifted relation, its probe count, the root calls and the relation checks are the S3 loop's.
- This is the rank-probe experiment's exactness result for `PIPE_FRAME` against `PIPE_COMPACT`, applied to the online target. That result rests on its tests and on four Tier 1 rank blocks that reproduced the S3 loop's relations and probes.

Observed locally. Whether a pair is emitted in the S3 order depends only on the target:

- Q0, s002, s004 and s005: every candidate is in the S3 order.
- s000, s001 and s003: every candidate is swapped.

So the hit resolution is exercised on about half of the targets. This matches the rank stage's roughly 47% swapped hits.

Local exactness and diagnostics (cloud-agent VM)

See smoke/README.md. In short:

- **Tests.** All 20 producer tests pass, including two new ones:
  - through `extract128` on 60 planted four-point targets, `PIPE_FRAME` and `PIPE_FRAME_S3PLANT` each return `PIPE_COMPACT`'s relation, probe count, root calls and relation checks;
  - the selector defaults to `s3`.

  The 4 tuned-rho tests also pass.
- **Runner, wrapper and verifier (O1, C1–C6, frozen binary).** All 34 rows are valid. On Q0 and points s000–s005, every IC process gives one scalar (also equal to the rT scalar, and replayed in the oracle). Every IC process also gives one relation, one set of x codes and intermediates, one probe count, one root-call count and one relation-check count. Q0 equals the archived Feistel identity (1,987,852 probes) in cS, cF and cFX.
- **Planted control.** cFX compared 6,759,354 candidates on 40,719 states, 4,169,754 of them in swapped order. It found no difference.
- **Shared rank process (K1, first smoke).** kP (seed 701, unit s031) reached full rank. Its factor-log table equals B's, it built one frame per attempt, it had 600 resolved hits (303 swapped), and it recovered the target.

- **Local timing (diagnostic only, on a shared VM at load 11 on 4 vCPUs).** The ratio of mean online times cS / cF over s000–s005 is 1.48. Per point it ranges 0.67–2.11, and the local A/A cF / cFaa is 0.956. The VM cannot resolve this, and the runner decides.

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, and one fresh process per run. Every IC arm runs the online-frame producer (`ic_online_onlineframe_r770`) with the 16-byte compact table over the folded index, lift_scan keys, the Feistel order and the prefetch pipeline. Every IC arm uses the verified cached factor-log table, except kP.

Arm	Kind	Online solve	Role
`cS`	cold	`s3`	**baseline**: the rank-probe producer's cold path
`cF`	cold	`frame`	**candidate**
`cFaa`	cold	`frame`	A/A of cF
`cFX`	cold	`frame_s3plant`	planted positive control
`rT`	rho	—	tuned rho rT770 (Fold/64/DP6), seed 900000 + point seed
`kP`	rank	`s3` (rank `frame`)	shared rank stage K, for the end-to-end secondary only

- **Panel.** 64 new public points (`inputs/cold_targets.json`, role `onlineframe-v1`, built by `make_targets.py`). All panels below are regenerated from the law and checked disjoint:
  - Q0 and B's panel;
  - scanorder-heldout, combined-primary-v1, probepipe-primary-v1, statefold-cold-v1, tablefilter-primary-v1, stacked-cold-v1, rankcut-cold-v1 and rankprobe-cold-v1;
  - the 64 `eval` points of n131-ref-walk-20261015;
  - the parallel branches' `indexlift-cold-v1` (origin/claude/n83-indexlift), `rotkey-cold-v1` and `k899-cold-v1`, each excluded over 128 seeds.

  The panel was also compared directly with every `inputs/*.json` in the three parallel worktrees, with no overlap. No scalar is known for any point.
- **Rank block.** There is one block, K1: one kP process, new rank seed 701, unit s031, placed between C32 and C33.
- **Segment order.** O1 (cS, cF and cFX on Q0; exactness only), then C1–C32, K1, C33–C64. The order within each segment is shuffled with new seeds. The time guard is 300 minutes; the job limit is 350.

Primary (pre-registered): the online ratio.

- R_on = the mean over the points of online_ms(cS), divided by the mean over the points of online_ms(cF).
- online_ms is the producer's interval from `target_query_begin` to `recovery_check_end`, for the one target in its cold process.
- Bootstrap: 10,000 draws, seed 20261020. The points are resampled, paired across cS and cF. The interval is the 95% percentile interval.
- **Improvement** iff the 95% lower bound of R_on exceeds 1; regression iff the upper bound is below 1; otherwise not separated.
- **Invalid** if any gate or control fails or any scheduled row is missing.

End to end (pre-registered secondary; never decides).

- E2E(c) = K + mean cold(c). The **same** shared rank stage K (the one kP process) is used in the numerator and the denominator.
- This is the index-lift design's shared-K reasoning (origin/claude/n83-indexlift, `experiments/koblitz-n83-indexlift-20261019/README.md`, "The rank stage: shared, and why"): the rank stage runs identical code over an identical index, so its cost is not a treatment effect. Separate rank processes per arm would only add their process drift. Here `KIC_ONLINE_SOLVE` is read only by the online scan.

- R_e2e = E2E(cS) / E2E(cF). Bootstrap seed 20261021: kP's 600 attempts are resampled, and the points are resampled in pairs.
- Expected value: about 1.0006.

How the verdict maps onto the success criterion. AGENTS.md, "Success criterion: iterative improvement of the index-calculus baseline", says:

> the success criterion is a verified, measured improvement over the current index-calculus baseline. Every such improvement is a success and is reported as one.

Under "Improvement" it adds:

> A stage-level win is reported as a stage improvement and becomes a pipeline improvement once it moves end-to-end `S`.

The frozen protocol (`success_criterion_reading`, `decision_rule`) therefore reads the verdict as follows:

- An online improvement (R_on lower bound above 1) with no end-to-end regression (R_e2e upper bound at or above 1) is a verified, measured improvement over the baseline, and so a success. It is reported as an online **stage** improvement. The candidate's online `S` and online gap to tuned rho replace the baseline row's.
- It is reported as a **pipeline** improvement only if R_e2e's lower bound also exceeds 1.
- An end-to-end regression (R_e2e upper bound below 1) blocks promotion.

Controls.

- A/A: online cF / cFaa and cold cF / cFaa must both be inside [0.97, 1.03].
- Planted: online cFX / cF must have its 95% lower bound above 1.

Gates.

- Every IC process:
  - uses the compact layout over the folded index;
  - matches B's index digests and has 29,878,182 entries;
  - records the arm's `online_solve`.
- Per point, across cS, cF, cFaa and cFX (and kP on its unit):
  - one scalar, the same as rT's, replayed in the oracle;
  - one relation (point indices, x codes and pinned intermediates);
  - one probe count, one root-call count and one relation-check count.
- Q0: the archived relation and scalar, with 1,987,852 probes, in cS, cF and cFX.
- Online counters:
  - cS: every counter is 0;
  - cF, cFaa and cFX: exactly one frame each;
  - only cFX checks states against the S3 loop, and it must check at least one.
- kP: full rank, `rank_solve` frame, one frame per attempt, and a factor-log table equal to B's.

Secondaries. None of them decides.

- Online ns per probe per arm.
- Online cS / cF by probe tercile, and as a geometric mean.
- Cold, setup, index build and peak RSS for cS / cF (expected 1).
- The counters.
- `S` per arm (online, cold, end to end) and delta `S`.
- The online and end-to-end gaps to rT770.

Power (`power_check.py`, `analysis/power.json`).

- The prior is `inputs/power_prior.json`, built by `make_power_prior.py` from run 39214027289's cR / cP online times on its 20 cold points. Those are two processes of the same S3 online code with identical probes, so their ratio is the null. The per-point A/A ratio was 0.992–1.018, SD 0.0065.
- The simulation used 400 runs per cell with 1,000 bootstrap draws each.
- Power is 1.00 at the minimal effect of interest, 1.05 online, for 16, 32, 64 and 128 points.
- The rejection rate at no effect is 0.000 at every size.

- The panel is 64 points, the frozen minimum. That is more than power needs, but it covers more targets with the exactness gates and the controls.
- Expected effect (not an input): about 1.4–1.6 online, from 221.4 against 141.5 ns per probe. That is about −100 ms end to end, a ratio of about 1.0006.

Runtime estimate. About 2.1 h including the build (up to about 2.8 h). A cold segment takes about 95 s, and K1 about 170 s.

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`make_online_source.py`, `make_rankprobe_source.py`, `make_rankcut_source.py`, `make_stacked_source.py`, `make_tablefilter_source.py`, `make_probepipe_source.py`, `make_combined_source.py`, `online/`, `lift2/`	the online-frame producer (the rank-probe derivation verbatim and digest-checked, plus `KIC_ONLINE_SOLVE`); tuned rho; the two-limb canonicalizer crate
`restore.py`, `check_log_table.py`	restore and digest-check crypto 770f6a69 and every local source; re-verify B's factor-log table
`make_targets.py`, `inputs/cold_targets.json`	the new panel (64 points)
`inputs/archived_identities.json`	archived Q0 identity
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 39214027289), simulation, result
`arm.py`, `run_schedule.py`	one arm per pinned process; the schedule runner
`verify_receipt.py`	gates, statistics, decision rule; `result.json`, `result_table.md`
`figures/design.*`	design diagram (TikZ source, PDF, PNG)
`make_figures.py`	result figure from `result.json`, after the run
`smoke/`	local diagnostics: tests, runner smoke, correctness, local timing
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`